JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class StringWriter

java.lang.Object
    java.io.Writer
        java.io.StringWriter

All Implemented Interfaces:

`Closeable`, `Flushable`, `Appendable`, `AutoCloseable`

```
public class StringWriter
extends Writer
```

A character stream that collects its output in a string buffer, which can then be used to construct a string.

Closing a `StringWriter` has no effect. The methods in this class can be called after the stream has been closed without generating an `IOException`.

Since:

`JDK1.1`

Field Summary

Fields inherited from class java.io.Writer

`lock`

Constructor Summary

Constructors

Constructor	Description
`StringWriter()`	Create a new string writer using the default initial string-buffer size.
`StringWriter(int initialSize)`	Create a new string writer using the specified initial string-buffer size.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`StringWriter`	`append(char c)`	Appends the specified character to this writer.
`StringWriter`	`append(CharSequence csq)`	Appends the specified character sequence to this writer.
`StringWriter`	`append(CharSequence csq, int start, int end)`	Appends a subsequence of the specified character sequence to this writer.
`void`	`close()`	Closing a `StringWriter` has no effect.
`void`	`flush()`	Flush the stream.
`StringBuffer`	`getBuffer()`	Return the string buffer itself.
`String`	`toString()`	Return the buffer's current value as a string.
`void`	`write(char[] cbuf, int off, int len)`	Write a portion of an array of characters.
`void`	`write(int c)`	Write a single character.
`void`	`write(String str)`	Write a string.
`void`	`write(String str, int off, int len)`	Write a portion of a string.

Methods inherited from class java.io.Writer

write

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

StringWriter

```
public StringWriter()
```

Create a new string writer using the default initial string-buffer size.

StringWriter

```
public StringWriter(int initialSize)
```

Create a new string writer using the specified initial string-buffer size.

Parameters:

`initialSize - The number of char values that will fit into this buffer before it is automatically expanded`

Throws:

`IllegalArgumentException - If initialSize is negative`

Method Detail

write

```
public void write(int c)
```

Write a single character.

Overrides:

`write in class Writer`

Parameters:

`c - int specifying a character to be written`

write

```
public void write(char[] cbuf,
                  int off,
                  int len)
```

Write a portion of an array of characters.

Specified by:

`write` in class `Writer`

Parameters:

`cbuf - Array of characters`

`off - Offset from which to start writing characters`

`len - Number of characters to write`

write

```
public void write(String str)
```

Write a string.

Overrides:

`write` in class `Writer`

Parameters:

`str - String to be written`

write

```
public void write(String str,
                  int off,
                  int len)
```

Write a portion of a string.

Overrides:

`write` in class `Writer`

Parameters:

`str - String to be written`

`off - Offset from which to start writing characters`

`len - Number of characters to write`

append

```
public StringWriter append(CharSequence csq)
```

Appends the specified character sequence to this writer.

An invocation of this method of the form `out.append(csq)` behaves in exactly the same way as the invocation

```
out.write(csq.toString())
```

Depending on the specification of `toString` for the character sequence `csq`, the entire sequence may not be appended. For instance, invoking the `toString` method of a character buffer will return a subsequence whose content depends upon the buffer's position and limit.

Specified by:

`append` in interface `Appendable`

Overrides:

`append` in class `Writer`

Parameters:

`csq - The character sequence to append. If csq is null, then the four characters "null" are appended to this writer.`

Returns:

`This writer`

Since:

`1.5`

append

```
public StringWriter append(CharSequence csq,
                           int start,
                           int end)
```

Appends a subsequence of the specified character sequence to this writer.

An invocation of this method of the form `out.append(csq, start, end)` when `csq` is not `null`, behaves in exactly the same way as the invocation

```
out.write(csq.subSequence(start, end).toString())
```

Specified by:

`append` in interface `Appendable`

Overrides:

`append` in class `Writer`

Parameters:

csq - The character sequence from which a subsequence will be appended. If csq is null, then characters will be appended as if csq contained the four characters "null".

start - The index of the first character in the subsequence

end - The index of the character following the last character in the subsequence

Returns:

This writer

Throws:

IndexOutOfBoundsException - If start or end are negative, start is greater than end, or end is greater than csq.length()

Since:

1.5

append

public StringWriter append(char c)

Appends the specified character to this writer.

An invocation of this method of the form out.append(c) behaves in exactly the same way as the invocation

```
out.write(c)
```

Specified by:

append in interface Appendable

Overrides:

append in class Writer

Parameters:

c - The 16-bit character to append

Returns:

This writer

Since:

1.5

toString

public String toString()

Return the buffer's current value as a string.

Overrides:

toString in class Object

Returns:

a string representation of the object.

getBuffer

```
public StringBuffer getBuffer()
```

Return the string buffer itself.

Returns:

StringBuffer holding the current buffer value.

flush

```
public void flush()
```

Flush the stream.

Specified by:

flush in interface Flushable

Specified by:

flush in class Writer

close

```
public void close()
           throws IOException
```

Closing a StringWriter has no effect. The methods in this class can be called after the stream has been closed without generating an IOException.

Specified by:

close in interface Closeable

Specified by:

close in interface AutoCloseable

Specified by:

close in class Writer

Throws:

IOException - If an I/O error occurs

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.